Table of Contents

Week 4 — Encapsulation

- The First Pillar: Protecting an Object's Own Data
- Lesson Overview
- Hour Plan
- Recap of Week 3 (5 min)
- Part 1 — What Is Encapsulation? (12 min)
  - Open with a question (2 min)
  - Definition
  - The Real-World Analogy: An ATM Machine
  - A Second Analogy: The Childproof Medicine Cap
  - The Two Parts of Encapsulation
- Part 2 — Public vs Private: Two Levels of Access (10 min)
  - Open with a question (2 min)
  - Definition
  - Real-Life Public vs Private Examples
  - The Encapsulation Recipe
- Part 3 — Read Access, Write Access & Validation (13 min)
  - Open with a question (2 min)
  - Definition
  - Three Real-World Validation Stories
  - Why Validation Matters — A Before/After Story
- Part 4 — Real-World Walkthrough: Before and After Encapsulation (10 min)
  - Sokha's Bank Account — Before Encapsulation (Open Access)
  - Sokha's Bank Account — After Encapsulation (Protected Access)
  - Discussion Prompt
- Part 5 — Java as Illustration (8 min)
  - Reading the Code as Concepts
- Test Checkpoint
- Common Misconceptions
- Extension Challenges
  - 🟢 Basic — Public or Private?
  - 🟡 Intermediate — Design the Protection

🔴 Advanced — When Encapsulation Gets Tricky

Connecting the Dots — Foundations + First Pillar

Lesson Summary

Homework

Week 4 — Encapsulation

The First Pillar: Protecting an Object's Own Data

Date: Wednesday, 24 June 2026 | **Time:** 6:30 PM – 8:30 PM | **Course:** Object-Oriented Programming Concepts

Lesson Overview

Item	Detail
Topic	Encapsulation — hiding data and exposing only safe, controlled access
Date & Time	Wednesday, 24 June 2026 · 6:30 PM – 8:30 PM
Duration	2 hour
Format	Online (Google Meet) — Concept explanation + real-world discussion + light illustration
Prerequisites	Week 2 (Classes & Objects), Week 3 (Attributes, Methods, State)
Focus	OOP concept understanding — Java used only as illustration
Outcome	Students can explain encapsulation in plain words using real-world analogies, distinguish private from public access, explain why validation matters, and recognize encapsulation in everyday systems

Hour Plan

```
00:00 – 00:05   Recap of Week 3 + Homework check
00:05 – 00:17   Part 1 – What Is Encapsulation?
00:17 – 00:27   Part 2 – Public vs Private: Two Levels of Access
00:27 – 00:40   Part 3 – Read Access, Write Access & Validation
00:40 – 00:50   Part 4 – Real-World Walkthrough: Before and After Encapsulation
00:50 – 00:58   Part 5 – Java as Illustration (light)
00:58 – 01:00   Checkpoint + Homework
```

> 💻 ***Google Meet tip:*** *Keep Week4_Summary_Examples.md open in a second tab — it has a fresh, different real-world example ready for every part of this lesson, in case a student needs a second angle.*

Recap of Week 3 (5 min)

Ask students verbally:

1. "What's the difference between an attribute and a method? Give an example from your own daily life."
2. "In the nametag story from last week — what was the mistake, and what was the fix?"
3. "What's the difference between a command method and a query method?"
4. "From homework — when Sokha's phone took 10 photos, which attributes changed? Was `takePhoto()` a command or a query? What about `checkStorage()`?"

> *Today we ask a new question: now that we know objects HAVE state and CAN change it — what stops that state from being changed incorrectly?*

Part 1 — What Is Encapsulation? (12 min)

Open with a question (2 min)

> *"Imagine your school kept every student's grades in one shared folder that ANY student could open and edit directly — no teacher, no registrar, no approval process. Just open the file and change any number you want.*
>
> *Would you trust a single grade in that school? Why or why not?"*

Let students react. They will immediately sense the danger — cheating, chaos, no accountability.

> *"This is exactly the problem when an object's information is left completely open. Today's lesson is about how OOP solves this — through a principle called **Encapsulation**."*

Definition

> ***Encapsulation*** *is the OOP principle of:*
> *1.* ***Bundling*** *an object's information (attributes) and its actions (methods) together in one place — we've already been doing this since Week 2*
> *2.* ***Hiding*** *that information from the outside world, so it can only be reached through approved, controlled actions*

The outside world can no longer reach in and touch an object's data directly. It must go through the object's own front door.

The Real-World Analogy: An ATM Machine

```
ATM MACHINE
──────────────────────────────────────────────────────────────
WHAT YOU CAN DO (the public, visible interface):
  → Insert your card
  → Enter your PIN
  → Select: Withdraw / Deposit / Check balance
  → Receive your cash

WHAT YOU CANNOT TOUCH (the hidden, private internals):
  → The vault holding the actual cash
  → The bank's internal transaction database
  → The PIN-checking logic
  → The wiring and mechanisms inside the machine

WHY DOES THIS DESIGN EXIST?
  → Protection – you cannot bypass the security checks
  → Integrity – the bank's rules are ALWAYS enforced, every time
  → Simplicity – you only ever see the buttons you need

THIS is encapsulation: a clear, controlled "front door" –
and everything behind it stays hidden and protected.
──────────────────────────────────────────────────────────────
```

A Second Analogy: The Childproof Medicine Cap

> *"Here's a different kind of 'protected access.' A medicine bottle has a childproof cap. You ARE allowed to open it — it's not locked away forever. But you must perform the correct action: squeeze and twist at the same time. A random push or pull will not open it.*
>
> *This shows something important: restricting access isn't always about saying 'no.' Sometimes it's about saying '**yes, but only if you do it the right way.**'"*

CHILDPROOF CAP	ENCAPSULATION CONCEPT
The pills are protected inside	The object's data is hidden inside (private)
You CAN access the pills	You CAN change the data — but only through approved actions (methods)
You must squeeze AND twist — the correct action — or it won't open	The action checks that your request is valid before allowing the change

---

The Two Parts of Encapsulation

```
PART 1 — BUNDLING (already done since Week 2)
  Attributes and methods that belong together live in one class.
  Student { name, age, major, introduce(), checkGrade() }

PART 2 — HIDING / RESTRICTING ACCESS (new this week)
  Attributes are marked PRIVATE — hidden from the outside world.
  The only way in or out is through approved PUBLIC actions.
```

Part 2 — Public vs Private: Two Levels of Access (10 min)

Open with a question (2 min)

> *"Think about your own house. Which parts are open for anyone to walk into — guests, delivery drivers, neighbors? Which parts are private — just for you and your family?"*

Students will say: living room/front yard = open to guests; bedroom, personal drawer, diary = private.

> *"Every house naturally has this split: some areas are **public**, some are **private**. Objects in OOP work exactly the same way."*

Definition

PUBLIC	PRIVATE
Anyone (any outside code) can directly reach and use it	Only the object itself – its own actions – can directly reach it
Outsiders walk right in	Outsiders must ask through an approved action instead

---

Real-Life Public vs Private Examples

SETTING	PUBLIC (open to anyone)	PRIVATE (restricted)
Your house	Living room, front yard	Bedroom, personal drawer
Your phone	Lock screen clock, time (visible without unlocking)	Photo gallery, messages (need to unlock first)
A school	School yard, notice board	Headmaster's office, student record files
A diary/journal	The cover (anyone can see you own a diary)	The pages inside (locked, just for you)
A bank	The lobby, ATM machines	The vault, the back office, the records room

> ***Forward note:*** *There is also a middle-ground idea — "family only" access — useful when one object is a specialized version of another (like a Student being a kind of Person). We will meet this properly in Week 6, when we study Inheritance. For now, every attribute is either fully private or fully public.*

The Encapsulation Recipe

```
ENCAPSULATION RECIPE — apply this to every class from now on:
──────────────────────────────────────────────────────────
Step 1: Make every attribute PRIVATE
        (hide all the object's information by default)

Step 2: Provide PUBLIC "read" actions
        (so outside code can safely ASK for information)

Step 3: Provide PUBLIC "write" actions
        (so outside code can safely REQUEST a change —
         but the object checks the request first)

Step 4: Keep any internal helper logic PRIVATE too
        (only expose what the outside world actually needs)
──────────────────────────────────────────────────────────
```

Part 3 — Read Access, Write Access & Validation (13 min)

Open with a question (2 min)

> *"Your diary is locked. A close friend asks: 'Can I read just this one page?' Another time, you want to ADD a new entry yourself. How do you allow safe, limited access — without handing over the key to everything?"*

> *"You'd let them read ONE page (a controlled, read-only peek), and when YOU add an entry, you'd check that it makes sense before writing it down (a controlled, checked update). These two ideas have names in OOP: a **getter** (read access) and a **setter** (write access, with checking)."*

Definition

```
GETTER (read access)                    SETTER (write access)
──────────────────────────────────────────────────────────────
A safe way to ASK for information       A safe way to REQUEST a change
Nothing changes — pure reading          The object checks the request
                                          is reasonable BEFORE accepting it
Example: "What's my current balance?"   Example: "Set my age to 16"
                                          → object checks: is 16 reasonable?
──────────────────────────────────────────────────────────────
```

> *A setter that accepts absolutely anything is not actually protecting anything. The real value of a setter is the **validation** — the check it performs before saying yes.*

Three Real-World Validation Stories

```
STORY 1 — The Childproof Cap (revisited)
  You CAN open the bottle — but only with the correct motion.
  → A setter CAN change the value — but only if it passes the check.

STORY 2 — The ATM Withdrawal Limit
  You ask the ATM to withdraw $10,000. Your account only holds $50.
  The ATM does NOT just hand over money it doesn't have.
  It REJECTS the request and explains why.
  → This is validation: checking a request BEFORE acting on it.

STORY 3 — The School Grading System
  A GPA must be between 0.0 and 4.0. It is mathematically impossible
  to score higher. If a registrar accidentally types "40.0" instead
  of "4.0", a properly protected system should immediately reject it
  — not silently accept an impossible number.
```

Why Validation Matters — A Before/After Story

```
WITHOUT VALIDATION
──────────────────────────────────────────────────────────────
A registrar accidentally types "40.0" for a GPA instead of "4.0".
The system accepts it without question — no check was ever made.
The student's transcript now shows an impossible GPA of 40.0.
```

```
WITH VALIDATION
──────────────────────────────────────────────────────────
The registrar types "40.0" for a GPA.
The system immediately checks: "Is this between 0.0 and 4.0?"
"40.0" fails the check.
The registrar sees an instant message: "Invalid GPA — must be
0.0 to 4.0" and is asked to re-enter it correctly.
The mistake is caught in SECONDS, not discovered months later.
──────────────────────────────────────────────────────────
The object protects its own integrity.
No outside mistake can silently corrupt its data.
```

Part 4 — Real-World Walkthrough: Before and After Encapsulation (10 min)

Sokha's Bank Account — Before Encapsulation (Open Access)

SCENARIO	WHAT COULD GO WRONG
Any code, anywhere in the program, can directly set the balance to any number	Balance becomes negative — an impossible situation in real banking
Any code can directly set the transaction count to anything	The account shows 9,999 transactions that never actually happened
There is no record of WHO changed the balance or HOW	No accountability. No way to trust the account's history

Sokha's Bank Account — After Encapsulation (Protected Access)

SCENARIO	WHAT HAPPENS NOW
Outside code tries to directly touch the balance	Rejected immediately — it is not even possible. Must use deposit() or withdraw() instead
Someone tries to withdraw more than the account holds	The withdraw() action checks the rule and refuses, explaining why

```
negative amount                          "is this positive?" and refuses
──────────────────────────────────────────────────────────────────────

The account now protects its own integrity.
No code anywhere else in the program can corrupt it — on purpose
or by accident.
```

Discussion Prompt

> *"Before this lesson, any code could write `acc.balance = -500000`. After encapsulation, what happens if you try? Why is this a meaningful improvement — not just a technical rule, but a real protection?"*

Part 5 — Java as Illustration (8 min)

> ***Reminder:** Java is just how we WRITE these concepts down. The concept came first — the code is only a translation.*

```java
// Encapsulated BankAccount — the concept made concrete in code

public class BankAccount {

    // PRIVATE attributes — hidden, protected, no direct outside access
    private String owner;
    private double balance;

    // PUBLIC read access (getter) — anyone can safely ASK for the value
    public double getBalance() { return balance; }
    public String getOwner()   { return owner; }

    // PUBLIC write access, WITH validation
    public void setOwner(String newOwner) {
        if (newOwner == null || newOwner.isBlank()) {
            System.out.println("Owner name cannot be empty.");
            return;                          // reject the bad request
        }
        owner = newOwner;
    }

    public void deposit(double amount) {
        if (amount ≤ 0) {
            System.out.println("Deposit must be a positive amount.");
            return;                          // reject the bad request
        }
```

```
        balance = balance + amount;     }                public void withdraw(double amount) {
        if (amount > balance) {
            System.out.println("Cannot withdraw more than your balance.");
            return;                              // reject the bad request        }
        balance = balance - amount;     }    }
```

> ***Notice:*** *there is no simple* `setBalance()`*. In real banking, you should never be able to directly "set" a balance to any number — you can only deposit or withdraw. Sometimes a setter isn't a plain one-line write — it is the business rule itself.*

```
public class Main {
    public static void main(String[] args) {

        BankAccount sokha = new BankAccount();
        sokha.setOwner("Sokha");

        // sokha.balance = -99999;   ← this line would not even compile!
        //                              'balance' is private - no direct access

        sokha.deposit(200.0);
        sokha.withdraw(5000.0);      // rejected - exceeds the balance

        System.out.println("Balance: " + sokha.getBalance());
    }
}
```

Reading the Code as Concepts

```
CONCEPT TRANSLATION TABLE
_____________________________________________________________

Java code                       OOP concept meaning
_________________________       ___________________________________

private balance                 Hidden - only this class can touch it
getBalance()                    Public read access - safely ask for it
deposit(amount)                 Public write access - request a change
if (amount ≤ 0) ... return;     Validation - the rule checked first
sokha.balance = -99999          Not even possible - proves the data
  (commented out)                is truly protected
_____________________________________________________________
```

Test Checkpoint

- ☐ Can explain encapsulation in one sentence using the ATM analogy or the school transcript story
- ☐ Can distinguish public from private using a real-world example (not ATM, not BankAccount)
- ☐ Can explain the difference between a getter and a setter in plain words
- ☐ Can explain why validation matters using one of the three real-world validation stories
- ☐ Can describe one thing that could go wrong WITHOUT encapsulation, and how encapsulation prevents it

---

Common Misconceptions

Misconception	Clarification
"Encapsulation just means hiding things for no reason"	It hides things so they can only be changed through *checked, approved* actions — protection, not secrecy for its own sake.
"A setter that accepts anything is still useful"	Not really — the real value of a setter is the validation check. A setter with no rules is no better than leaving the data public.
"Private means nobody can ever change the data"	No — it means outside code cannot touch it *directly*. The object's own public actions can still change it, safely.
"Public and private are about secrecy"	They are about *responsibility*. Public = the object's promise to the outside world. Private = the object's own internal business.
"Validation is extra, optional work"	Validation is the entire point of a setter. Without it, an object cannot protect its own integrity.

Extension Challenges

🟢 Basic — Public or Private?

For each item below, decide if it should be **Public** or **Private**, and explain why in one sentence.

1. A student's current GPA (should outside code be able to set it to any number directly?)
2. A library's opening hours (should everyone be able to see this freely?)
3. A safe's combination code
4. A restaurant's menu (visible to customers)
5. A restaurant's secret recipe

🟡 Intermediate — Design the Protection

Pick a real object from your daily life — a school locker, a social media account, or a water tank with a tap.

- List 3 attributes. Decide: should each be private or public?
- Design 1 getter (read access) and 1 setter (write access, with a validation rule)
- Describe in plain English: what bad situation does your validation rule prevent?

🔴 Advanced — When Encapsulation Gets Tricky

A `Temperature` object stores a value in Celsius. It is impossible for any real temperature to go below **−273.15°C** (absolute zero — the coldest physically possible temperature).

- Design the validation rule for a setter that updates this temperature
- Write a short explanation (3–5 sentences): why is it better for the `Temperature` object itself to enforce this rule, rather than relying on every single piece of code that uses temperatures to remember to check it manually?

Connecting the Dots — Foundations + First Pillar

```
WHAT WE HAVE LEARNED SO FAR
──────────────────────────────────────────────────────────────

WEEK 1 — What is OOP?
  → OOP organizes code as interacting objects
  → 4 pillars: Encapsulation, Inheritance, Polymorphism, Abstraction

WEEK 2 — Classes & Objects
  → Class = template/blueprint; Object = real instance with real data
  → Every object holds its own independent copy of data

WEEK 3 — Attributes, Methods & Object State
  → Attributes = what an object knows; Methods = what it can do
  → Persistent vs given-from-outside vs temporary information
  → Command methods change state; query methods only read it
```

```
WEEK 4 — Encapsulation
  → Hide an object's data (private) — outsiders cannot reach it directly
  → Expose only safe, controlled access points (public getters/setters)
  → Validate every requested change BEFORE accepting it
  → The object now protects its OWN integrity

NEXT: Week 5 — Constructors
  → We will build objects that are ALREADY valid and complete
    the moment they are created
  → No more separate setup calls needed after creation
──────────────────────────────────────────────────────────────────────
```

Lesson Summary

```
TODAY'S KEY TAKEAWAYS
──────────────────────────────────────────────────────────
1. Encapsulation = bundling data + behavior together,
   then HIDING that data from direct outside access.

2. Private = only the object itself can directly touch it.
   Public = anyone (any outside code) can directly use it.

3. A getter provides safe, READ-ONLY access to private data.
   A setter provides safe, CHECKED write access — it can
   reject a request that doesn't make sense.

4. Validation is the heart of a setter's value. Without it,
   a setter is no better than leaving the data public.

5. Encapsulation lets an object protect its OWN integrity —
   no outside code, by accident or on purpose, can corrupt it.

6. This is not about secrecy. It is about responsibility:
   the object owns and protects its own information.
──────────────────────────────────────────────────────────
```

Homework

Continue your `Smartphone` design from Weeks 2–3. Apply encapsulation:

1. Decide: which attributes should be **PRIVATE**? (Hint: probably all of them — `batteryPercent`, `storageUsedGB`, etc.)

2. Design a **getter** for checking the battery level — pure read access, changes nothing
3. Design a **setter-style action** for charging the phone, with a **validation rule**: battery cannot go above 100% or below 0%
4. Write a short before/after comparison (3–5 sentences): what could go wrong if any outside code could directly set `batteryPercent` to `500` or `-50`? What does your validation rule prevent?

Optional bonus (only if you want extra practice): write this as actual Java code, following today's illustration as a guide.

Bring your design to Week 5 — we will use it to explore how to build a *complete, valid* Smartphone object in a single step, instead of setting each attribute one at a time after creation.

Week 4 of 16 | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com